Catalyst Router

Workspaces, Data Channels, and Data Control

How organizations onboard, connect sensors, and control data flow

Version 1.0

March 2026

Executive Summary

Catalyst Router gives organizations a way to connect sensors, services, and data sources across locations — and control exactly how that data flows between them. Rather than routing all traffic through a central hub, Catalyst places a lightweight router at each site. These routers peer with each other directly, advertising what data is available and building efficient paths automatically.

This document walks through the journey of onboarding an organization onto Catalyst: from provisioning a workspace, to deploying the first router node, to registering sensors as data channels, to establishing peering between sites. Along the way, it explains the terms, the moving parts, and the controls that keep data flowing only where it should.

Table of Contents

1. Key Terms . . . 2
2. The Onboarding Journey . . . 2
   2.1. Step 1: Provisioning a Workspace . . . 3
   2.2. Step 2: Deploying the First Node . . . 3
      2.2.1. Bootstrap sequence . . . 3
   2.3. Step 3: Connecting a Sensor (Registering a Data Channel) . . . 4
      2.3.1. Protocol support . . . 4
   2.4. Step 4: Deploying More Nodes and Establishing Peering . . . 4
      2.4.1. Minting peer tokens . . . 5
      2.4.2. Registering peers . . . 5
      2.4.3. Automatic route propagation . . . 5
   2.5. Step 5: Data Flows — Sensor to Consumer . . . 6
      2.5.1. The traffic path . . . 6
      2.5.2. What the consumer sees . . . 6
3. Data Control . . . 7
   3.1. Explicit Peering as Coarse-Grained Control . . . 7
   3.2. Role-Based Authorization . . . 7
   3.3. Workspace-Level Fleet Management . . . 8
   3.4. Network Isolation . . . 8
4. Looking Forward . . . 8
   4.1. Per-Peer Route Policies . . . 8
   4.2. Cross-Organization Partnerships . . . 8
   4.3. Data Classification Labels . . . 9
   4.4. SDK Integration . . . 9
   4.5. SaaS Management Plane . . . 9
5. Summary . . . 9

Key Terms

Before diving into the onboarding journey, here are the terms you'll encounter throughout this document.

Term	Definition
Workspace	The top-level boundary for a Catalyst deployment. A workspace represents a domain (e.g., `somebiz.local.io`) and contains the nodes, data channels, and peering relationships that belong to an organization. Licensing, fleet management, and identity are all scoped to the workspace.
Node	A single Catalyst Router deployment. Each node runs a set of co-located services — an orchestrator, a proxy, an authentication service, and optionally a gateway — packaged as a "Core Pod." Nodes are the building blocks of the mesh: you deploy one at each site where data originates or is consumed.
Data Channel	A named endpoint registered with a node. A data channel represents something that produces or consumes data — a Zenoh sensor feed, a TAK server, a GraphQL API, a video stream. Each data channel has a name, a protocol type, and an endpoint address. When a data channel is registered, Catalyst advertises it to peer nodes so that consumers elsewhere in the mesh can reach it.
Peer	A trust relationship between two nodes. Peering is always explicit — one node's administrator must register the other as a peer and provide an authentication token. Once peered, nodes exchange route information automatically, telling each other what data channels are available and how to reach them.
Data Custodian	The role responsible for managing data channels on a node. Data custodians register and remove data channels, controlling what data their site makes available to the mesh.
Route	The internal representation of a data channel as it propagates through the mesh. When Node A registers a data channel, that registration becomes a route that peer nodes install in their routing tables. Routes carry path information so the system can detect loops and select the shortest path.
Partnership	A peering relationship between nodes belonging to *different* organizations. Partnerships enable cross-organization data sharing with policy controls over which data channels are visible to the partner.

Table 1: Catalyst terminology.

The Onboarding Journey

This section walks through the process of bringing an organization onto Catalyst, from initial workspace provisioning through to live data flowing between sites. We'll use a concrete example throughout: a defense organization deploying Catalyst to connect sensor feeds across three locations.

Step 1: Provisioning a Workspace

Every Catalyst deployment begins with a workspace. Orbis provisions a workspace for the customer, which establishes:

- A **domain** that identifies the organization (e.g., `somebiz.local.io`). All nodes within the workspace share this domain as their identity boundary.
- **Licensing** managed at the workspace level, covering the fleet of nodes and their capabilities.
- **Root identity** — the workspace's authentication service generates the cryptographic keys used to issue tokens and verify trust within the organization.

For our example, the organization receives a workspace with the domain `somebiz.local.io`. This domain will appear in every node's identity and in the tokens that authenticate peer connections.

Step 2: Deploying the First Node

With a workspace provisioned, the organization deploys its first Catalyst Node. A node is a self-contained unit — everything it needs runs together as a "Core Pod":

Component	What It Does
Auth Service	Manages cryptographic keys and issues JWT tokens. On first startup, it generates a system admin token that bootstraps the rest of the node.
Orchestrator	The brain of the node. Manages data channel registration, peer sessions, and route propagation. Configures the proxy for traffic forwarding.
Envoy Proxy	Handles actual data traffic — incoming connections from local services and outgoing connections to peer nodes. Supports any TCP or UDP protocol.
Gateway	Optional. Provides GraphQL federation for HTTP-based services, stitching multiple APIs into a single endpoint.

Table 2: Components of a Catalyst Node.

Bootstrap sequence

Deploying a node follows a deliberate startup order:

1. **Auth service starts first.** It generates its cryptographic keys and mints a system admin token — the "master key" for this node. This token is logged to the console for the operator to capture.
2. **Orchestrator starts with the system token.** Using this token, the orchestrator mints itself a `NODE` token — a scoped credential that proves it is an authorized component of this workspace. From this point, the orchestrator can register data channels, create peers, and manage routes.
3. **Proxy starts and connects to the orchestrator.** The orchestrator feeds configuration to the proxy via xDS (the same protocol that Envoy uses in large-scale service meshes). As data channels and peers are added, the proxy automatically creates the right listeners and routes.

At this point, the node is running but isolated — it has no data channels and no peers. It's a router with nothing to route.

Step 3: Connecting a Sensor (Registering a Data Channel)

Now the organization connects its first data source. In our example, this is a Zenoh router that aggregates sensor feeds — military unit positions in NATO 2525C format (Cursor on Target), published by field emulators at locations like Wiesbaden and Virginia.

The Zenoh router runs alongside Node A and publishes data on TCP port 7447. To make this data available through the Catalyst mesh, the operator registers it as a data channel:

```shell
catalyst node route create "zenoh-router" \
  "http://zenoh-router:7447" \
  --protocol tcp
```

This single command does several things:

1. **Registers the data channel** with the orchestrator's routing table. The name `zenoh-router` becomes the identifier that other nodes use to find this service.
2. **Creates an Envoy ingress listener.** The proxy is automatically configured to accept incoming TCP connections for this data channel and forward them to the Zenoh router.
3. **Advertises the route to peers.** Once peering is established (next step), this data channel will be announced to every connected node.

Protocol support

Catalyst supports any protocol that runs over TCP or UDP. The `--protocol` flag tells the system how to handle the traffic:

Protocol	Behavior
tcp	Raw TCP passthrough. The proxy forwards bytes without inspection. Used for Zenoh, video streams, TAK, and custom protocols.
http	HTTP routing with header-based matching. The proxy can inspect HTTP headers for routing decisions.
http:graphql	GraphQL federation. The gateway stitches this service's schema into a unified supergraph, enabling cross-service queries.
http:grpc	gRPC with HTTP/2 framing. The proxy handles gRPC health checking and load balancing.

Table 3: Supported protocol types.

The key insight: Catalyst doesn't require applications to speak a specific protocol. A legacy sensor that speaks raw TCP works just as well as a modern GraphQL API. The mesh handles the routing; the application handles its own protocol.

Step 4: Deploying More Nodes and Establishing Peering

A single node with a data channel is useful, but the power of Catalyst emerges when nodes connect. The organization deploys two more nodes — Node B as a transit relay and Node C at a consumer site — and peers them into a chain: A ↔ B ↔ C.

Minting peer tokens

Peering in Catalyst is always explicit. Before two nodes can connect, each side must issue a token that authorizes the other. This is a deliberate design choice: no node can join the mesh without the administrator's knowledge and consent.

The process:

1. **Node A's auth service mints a token for Node B.** This token says "Node B is authorized to connect to Node A as a peer."
2. **Node B's auth service mints a token for Node A.** This is the reverse — "Node A is authorized to connect to Node B."
3. Both tokens are scoped to the workspace domain (`somebiz.local.io`) and carry the `NODE` principal, which grants permission to exchange routes.

Registering peers

With tokens in hand, the operator registers each peer:

```shell
# On Node A: register Node B as a peer
catalyst node peer create "node-b.somebiz.local.io" \
  "ws://orch-b:3000/rpc" \
  --domains "somebiz.local.io" \
  --peer-token "$TOKEN_FROM_B_FOR_A"

# On Node B: register Node A as a peer
catalyst node peer create "node-a.somebiz.local.io" \
  "ws://orch-a:3000/rpc" \
  --domains "somebiz.local.io" \
  --peer-token "$TOKEN_FROM_A_FOR_B"
```

The same process is repeated for B ↔ C. Once both sides register each other, the nodes automatically:

1. Open a WebSocket connection and exchange OPEN messages.
2. Negotiate a hold timer (how long to wait before considering the peer unreachable).
3. Begin exchanging routes — Node A tells Node B about the `zenoh-router` data channel, and Node B relays that information to Node C.

Automatic route propagation

This is where the BGP-inspired design pays off. The operator didn't need to tell Node B or Node C about the `zenoh-router` data channel. The route *propagated automatically* through the peering chain:

1. Node A advertises `zenoh-router` to Node B with the path `[node-a]`.

2. Node B installs the route and re-advertises it to Node C with the path `[node-b, node-a]`.
3. Node C installs the route. It now knows that `zenoh-router` is reachable through Node B, which reaches it through Node A.

Each node's proxy is automatically configured with the right listeners:

Node	Listener	Purpose
A	`ingress_zenoh-router`	Accepts incoming TCP and forwards to the local Zenoh router.
B	`egress_zenoh-router_via_node-a`	Forwards TCP to Node A's proxy.
C	`egress_zenoh-router_via_node-b`	Forwards TCP to Node B's proxy.

Table 4: Envoy listeners created automatically by route propagation.

Step 5: Data Flows — Sensor to Consumer

With peering established and routes propagated, the TAK adapter consumer on Node C can now reach the Zenoh sensor feed on Node A — even though the two nodes have no direct connection.

The traffic path

```
tak-adapter-consumer (Node C)
        |
        | connects to localhost:10000 (local Envoy egress)
        v
  envoy-proxy-c ——— mesh network ———> envoy-proxy-b
                                              |
                                              | forwards to Node A
                                              v
                                        envoy-proxy-a
                                              |
                                              | forwards to local service
                                              v
                                        zenoh-router:7447 (Node A)
```

The consumer's Zenoh client connects to its local Envoy proxy as if the Zenoh router were running locally. The Envoy mesh handles the multi-hop forwarding transparently. The Zenoh protocol passes through unmodified — the proxies treat it as raw TCP, preserving every byte.

What the consumer sees

From the TAK adapter consumer's perspective, it's connecting to a Zenoh router at `tcp/envoy-proxy-c:10000`. It has no knowledge of Node A or Node B. It receives NATO 2525C CoT events — infantry positions from Wiesbaden, armor movements from Virginia — as if the sensor were on its local network.

This is the core value proposition: **the mesh makes remote data channels feel local.** Applications don't need to know about the topology. They connect to their local node, and Catalyst handles the rest.

Data Control

Connecting nodes and flowing data is only half the story. The other half is **control** — ensuring that data flows only where it should, that the right people can manage data channels, and that organizations retain sovereignty over their information.

Explicit Peering as Coarse-Grained Control

The most fundamental control in Catalyst is that peering is never automatic. Every connection between nodes requires:

1. An administrator to decide that two nodes should peer.
2. Each node's auth service to mint a token for the other.
3. Both sides to register each other as peers.

This means an organization always knows exactly which nodes can see its data channels. There are no surprise connections, no auto-discovery that might expose data to unintended recipients. If Node A hasn't explicitly peered with Node C, Node C will never learn about Node A's data channels — even if both are peered with Node B.

Wait — didn't we just say routes propagate through transit nodes? Yes, but only through the peering chain. Node B relays Node A's routes to Node C because the administrator established both peering relationships. If the administrator peers A ↔ B but not B ↔ C, the routes stop at Node B.

Role-Based Authorization

Within a node, Catalyst uses Cedar — a policy language developed by Amazon — to control who can perform which operations. The authorization model defines five roles:

Role	Capabilities
Admin	Full control. Can manage peers, data channels, tokens, and policies.
Node Custodian	Manages peering relationships. Can create and remove peers, but cannot modify data channels directly.
Data Custodian	Manages data channels. Can register and remove data channels, controlling what data the node makes available.
Node	The node's own identity. Used for peer-to-peer communication (iBGP route exchange, gateway configuration).
User	Read-only access. Can view the list of peers and data channels but cannot modify anything.

Table 5: Authorization roles.

This separation of concerns means the person who manages network topology (Node Custodian) is not necessarily the same person who decides what data is shared (Data Custodian). In practice, this reflects how many organizations operate: network engineers handle connectivity, while data owners handle what flows over those connections.

Workspace-Level Fleet Management

The workspace provides the organizational boundary for all of this. Within a workspace:

- All nodes share a common domain and trust root.
- Tokens issued by any node's auth service are valid across the workspace.
- The licensing model is managed at the workspace level.

Orbis provisions the workspace and delivers it to the customer. In the current model, this includes standing up the initial identity infrastructure and configuring the domain. In the future, a SaaS management plane will provide self-service workspace provisioning, fleet-wide visibility, and centralized policy management.

Network Isolation

Catalyst nodes enforce network isolation by design. In the Zenoh demo, each node runs in its own Docker network (`stack-a`, `stack-b`, `stack-c`). Only the orchestrators and Envoy proxies join a shared `mesh` network. Local services — the Zenoh router, the TAK adapters — are invisible to the mesh. They communicate only with their local node.

This mirrors real-world deployments where nodes run in separate VPCs, separate facilities, or even air-gapped networks with controlled cross-domain links. The mesh traffic between nodes is the only path that data travels between sites.

Looking Forward

The controls described above — explicit peering, role-based authorization, workspace boundaries — represent the foundation. Several capabilities are planned that will give organizations finer-grained control over their data.

Per-Peer Route Policies

Today, when two nodes peer, all data channels are shared between them. In a future release, administrators will be able to define **export policies** per peer — for example, "share the `zenoh-router` data channel with Node B, but not the `classified-feed` channel." This will use the same Cedar policy engine, extending it from operation-level authorization to route-level filtering.

Cross-Organization Partnerships

Internal peering (within a workspace) is the current focus. The next milestone introduces **external peering** — partnerships between nodes in different workspaces, belonging to different organizations. Partnerships will include:

- Independent trust roots (each organization maintains its own certificate authority).
- Explicit route export policies (only designated data channels are visible to the partner).

- Token exchange via JWKS discovery rather than shared secrets.

This mirrors how the legacy Catalyst platform handled partnerships: organizations explicitly invited each other, and data custodians controlled which channels were shared. The new system preserves this intent while replacing the centralized gateway with decentralized peering.

Data Classification Labels

A planned enhancement to the data channel schema will support **classification labels** — metadata tags that describe the sensitivity or handling requirements of a data channel. Combined with per-peer route policies, this will enable rules like "never export data channels labeled `SECRET` to external partners" or "only route `CUI` data through nodes in approved regions."

SDK Integration

For organizations building applications on top of Catalyst, the SDK will provide a programmatic interface for data channel management. Rather than using the CLI, applications will be able to register themselves as data channels, discover available channels, and subscribe to route changes through a simple API. This "easy button" approach will make Catalyst integration a few lines of code rather than a deployment task.

SaaS Management Plane

The workspace concept is designed to scale beyond self-hosted deployments. A future SaaS management plane will provide:

- Self-service workspace provisioning for new customers.
- Fleet-wide dashboards showing node health, data channel status, and peering topology.
- Centralized policy management that pushes Cedar policies to all nodes in a workspace.
- Usage metering and license enforcement.

Summary

Catalyst Router gives organizations a decentralized way to connect data sources across locations while maintaining control over how data flows. The journey from zero to live data follows a clear path:

1. **Workspace provisioning** — Orbis establishes the organizational boundary, domain, and identity.
2. **Node deployment** — each site gets a self-contained router that bootstraps its own authentication and proxy.
3. **Data channel registration** — sensors, services, and feeds register with their local node using any TCP or UDP protocol.
4. **Peering** — administrators explicitly connect nodes, and routes propagate automatically through the mesh.
5. **Data flow** — consumers connect to their local node and access remote data channels as if they were local.

At every step, control is explicit: peering requires mutual consent, operations require authorized roles, and data channels are visible only through the peering topology that administrators build. As the platform evolves, per-peer policies, cross-organization partnerships, and data classification will add progressively finer-grained control — always with the principle that data flows only where the organization intends.